

CS336复习笔记 1

Introduction

实验都做了什么？

- 参考原文档，可以得知，共有四个部分
- 1. Byte-pair encoding (BPE) tokenizer (§2)
 2. Transformer language model (LM) (§3)
 3. The cross-entropy loss function and the AdamW optimizer (§4)
 4. The training loop, with support for serializing and loading model and optimizer state (§5)
- 依次为
 - 实现一个BPETokenizer，可以将文本中出现频率最高byte-pair合并为一个byte，并且根据这种合并关系得到一个byte向id映射的表，最终根据这个表对文本进行编码和解码
 - 利用基本的torch库函数，实现一个最简版本的transformer，其中每个模块都需要自己实现
 - 训练模型所需要的工具
 - 实现训练循环和保存、加载模型的工具
- 主要的开发难度集中在前两个部分中

BPE Tokenizer的实现（1） - Train BPE部分

原理

BPE是一种介于char-level和word-level的编码方式

- 首先从char-level来看，每个字符用一个编码，自然可以表示正常情况下的所有文本，但是这种方法会大量占用空间，同时其实没有起到什么实际意义上的作用，只不过是把字符逐个转写成了数字
- 其次看word-level编码，每个单词应用一个编码，固然可以节约空间，但是针对训练过程中没有出现过的单词将无法编码

因此寻找一个折中点，建立在对所有字符编码的基础上，将最常出现的字符合并为一个“字符”（这里事实上是通过bytes数据类型实现的，合并为了一个二进制字符串），并保存合并顺序以方便编码。

在实际实现过程中，需要注意使用正则表达式来将特殊标识从文本中分离出去，以及由于文本往往很大，因此不能每次都从头到尾扫一遍来更新，所以需要保存过程中的统计信息来不断重复寻找最高频字符对。主要维护两个映射关系`{token:[frequency,split]}`和`{byte_pair:[frequency,tokens]}`

除此之外，原文档给出了将文本预分词为tokens的正则表达式，在正式进行 BPE 合并前，先对文本进行初步的分词或边界划分，确保 BPE 不会跨越语义或结构上不应合并的边界（如空格、标点、中文词、代码符号等）。

思考

- 如果提问到的话，这里的实现思路应该是重点，但是细节应该不重要，个人感觉太基础了
- 应该不至于手撕

实现

初始化词库并进行预处理，首先将256个字符转为bytes并导入vocab，将special_tokens进行encode并导入vocab

```
1      # 初始化vocab
2      # 256个单字节 + special tokens
3      # 注意是bytes([i])，传入一个可迭代对象，否则会产生i个0
4      vocab: dict[int, bytes] = {i: bytes([i]) for i in range(256)}
5      # 导入特殊符号
6      special_tokens_vocab = {256 + i: token.encode("utf-8") for i,
7                             token in enumerate(special_tokens)}
7      # 合并
8      vocab.update(special_tokens_vocab)
```

编译pre_tokenize和查找special_tokens的正则表达式，实测这一部分其实不是很耗时，但是严格来看的话最好在初始化过程中编译

```

1      # 编译pre_token过程中要使用的正则表达式
2      pre_token_re = re.compile(PAT)
3
4      # 编译分离特殊符号的正则表达式
5      special_pat = "|".join(re.escape(t) for t in special_tokens) #
    注意转义，最终是形如 a|b|c 的形式
6      special_re = re.compile(special_pat) if special_pat != "" else
    None

```

准备保存以下数据

- 每一个token的频数和划分方式， token_dict
 - 保存频数是一个省空间的trick，当然事实上其实没有省很多
- bytes对的频数以及位于哪些token中， pair_dict，用于在merge过程中查询并进行更新
- merges
 - 按顺序保留每次选中的bytes-pair

```

1      pair_dict = PairDict()
2      token_dict = TokenDict()
3      merges: list[tuple[bytes, bytes]] = []

```

为了方便过程中调用，为dict实现了一些方法，主要是包装一些访问方法以及错误处理

```

1  class TokenDict:
2      def __init__(self):
3          # 记录当前所有token的频次，注意token是bytes类型
4          self.token2count = {}
5          # 记录每个token当前的划分模型，初始时都是单个字符
6          self.token2splits: dict[bytes, list[bytes]] = {}
7
8  class PairDict:
9      def __init__(self):
10         self.pair2count = {}
11         self.pair2tokens: dict[tuple[bytes, bytes], set[bytes]] =
    {}
12
13     def add_pair(self, pair: tuple[bytes, bytes], count: int,
    token: bytes):
14         try:
15             self.pair2count[pair] += count
16             self.pair2tokens[pair].add(token)
17         except KeyError:

```

```

18         self.pair2count[pair] = count
19         self.pair2tokens[pair] = {token}
20
21     def discard_pair(self, pair: tuple[bytes, bytes], count: int,
22 token: bytes):
23         try:
24             self.pair2count[pair] -= count
25             self.pair2tokens[pair].discard(token)
26         except KeyError:
27             # 抛给上层处理
28             raise KeyError
29
30     def __getitem__(self, pair: tuple[bytes, bytes]) -> tuple[int,
31 set[bytes]]:
32         return self.pair2count[pair], self.pair2tokens[pair]
33
34     def get_max_pair(self) -> tuple[bytes, bytes]:
35         max_freq = max(freq for freq in self.pair2count.values())
36         max_pairs = [pair for pair, freq in self.pair2count.items()
37 if freq == max_freq]
38         # 返回其中字典序最大的pair
39         return max(max_pairs)

```

首先使用find_chunk_boundaries对文本进行预处理，将文本划分为数个chunk，注意需要将chunk解码为str才能应用正则表达

```

1     # 初始化token_dict和pair_dict
2     with open(input_path, "rb") as f:
3         num_chunks = os.cpu_count()
4         boundaries = find_chunk_boundaries(f, num_chunks, b"
<|endoftext|>")

```

分别处理每个chunk，首先读取全部文本进行dict的初始化过程，之后就不需要再扫描文本文件，可以提升效率

- 注意要对chunk中的换行符进行替换处理 `chunk = re.sub(r"\r\n?", "\n", chunk)`
-
- 使用special_tokens的正则表达式将chunk划分为数个part，形成一个parts列表，如果没有特别token，就只有一个part
- 对于每个part
 - 使用pre_tokenize正则表达式将part划分为数个pre_token，使用迭代器每次读入一个token进行处理

- 对于每个pre_token
 - 首先编码为bytes
 - 在pre_token_dict中更新值域：pre_token_dict[][0] = 频数, pre_token_dict[][1] = 划分方式，也就是按照bytes划分的列表
 - 遍历bytes划分方式，更新pair_dict中每个bytes对的值域：pair_dict[][0] = 频数, pair_dict[][1] = 出现该bytes对的token列表
 - 注意这里有个小区别：pre_token_dict中存储的是bytes列表，而pair_dict中存储的是一个set,保存该pair所在的token，用于在后续合并时更新计数
 - 注意通过trycatch捕获键错误

[illegible]

```

27         token_dict.token2splits[token_by] =
    token_split
28
29         # 更新pair_dict, 统计token_by中相邻字符对的频次
30         token_split = token_dict.token2splits[token_by]
31         for adj_pair in zip(token_split[:-1],
    token_split[1:]):
32             pair_dict.add_pair(adj_pair, 1, token_by)

```

完成初始化统计后，开始尝试合并，统计len(vocab)直到大小达到指定数量

```

1 | while len(vocab) < vocab_size:

```

开始循环merge

- 寻找最大频率的bytes对
- 找到这些bytes对中字典序最大的bytes对，将该bytes对添加到merges列表中
- 将该bytes对合并为一个新的token，加入vocab
- 根据pair_dict所存储的token列表，找到该bytes对所在的tokens，进行更新，注意这里的数据类型是set，需要转为list进行遍历
 - 在token_dict中找到每一个token，得到其频数tok_freq以及划分方式
 - 统计旧划分方式的相邻bytes对，根据merge的bytes对，统计新划分方法下的相邻bytes对
 - 更新pre_token_dict中该token的划分方式
 - 统计两种划分方式下的bytes对，进行计数，在pair_dict中进行更新
 - pair_dict的频数部分减去旧划分方式下的count * tok_freq，并在set部分中删除该token
 - pair_dict的频数部分加上新划分方式下的count * tok_freq，并在set部分中添加该token
 - 注意通过trycatch捕获键错误

```

1 | max_pair = pair_dict.get_max_pair()
2 | merged_pair = max_pair[0] + max_pair[1]
3 | merges.append(max_pair)
4 | vocab[len(vocab)] = merged_pair
5 | # 对所有包含该pair的token进行更新
6 | # 首先根据pair_dict找到包含该pair的token
7 | # 注意这里使用pop而不是直接访问，是因为后续不再需要访问该pair了
8 | # 但是由于key同时会被pop，因此后续访问时可能会出现KeyError
9 | tokens_to_update = pair_dict.pair2tokens.pop(max_pair)

```

```

10         # 最后，统计每个token中该pair的出现次数，注意根据划分模式进行统计
11         for token in tokens_to_update:
12             token_freq = token_dict.token2count[token]
13             old_split = token_dict.token2splits[token]
14             new_split = get_new_split(old_split, max_pair,
merged_pair)
15             # 统计旧划分模式中该pair的出现次数
16             old_adj_pairs = list(zip(old_split[:-1],
old_split[1:]))
17             new_adj_pairs = list(zip(new_split[:-1],
new_split[1:]))
18             # 更新token_dict中的划分方式
19             token_dict.token2splits[token] = new_split
20
21             old_cnt = Counter(old_adj_pairs)
22             new_cnt = Counter(new_adj_pairs)
23
24             # 更新计数
25             for pair, count in old_cnt.items():
26                 # count是该pair在该token中出现的次数，token_freq是该token
在语料库中出现的次数
27                 # 二者的乘积就是该pair在语料库中出现的次数
28                 # 更新非合并对的频数和位置
29                 try:
30                     # 这里可能会出现KeyError，因为某些pair可能已经被合并掉
了
31                     pair_dict.discard_pair(pair, count *
token_freq, token)
32                 except KeyError:
33                     # 已经作为max_pair被合并掉了
34                     # pair2tokens会有KeyError
35                     continue
36
37             for pair, count in new_cnt.items():
38                 # 与上述类似
39
40                 pair_dict.add_pair(pair, count * token_freq, token)

```

完整代码

```

1
2 def find_chunk_boundaries(
3     file: BinaryIO,
4     desired_num_chunks: int,
5     split_special_token: bytes,

```

```

6 ) -> list[int]:
7     """
8     Chunk the file into parts that can be counted independently.
9     May return fewer chunks if the boundaries end up overlapping.
10    """
11    assert isinstance(split_special_token, bytes), "Must
represent special token as a bytestring"
12
13    # Get total file size in bytes
14    file.seek(0, os.SEEK_END)
15    file_size = file.tell()
16    file.seek(0)
17
18    chunk_size = file_size // desired_num_chunks
19
20    # Initial guesses for chunk boundary locations, uniformly
spaced
21    # Chunks start on previous index, don't include last index
22    chunk_boundaries = [i * chunk_size for i in
range(desired_num_chunks + 1)]
23    chunk_boundaries[-1] = file_size
24
25    mini_chunk_size = 4096 # Read ahead by 4k bytes at a time
26
27    for bi in range(1, len(chunk_boundaries) - 1):
28        initial_position = chunk_boundaries[bi]
29        file.seek(initial_position) # Start at boundary guess
30        while True:
31            mini_chunk = file.read(mini_chunk_size) # Read a
mini chunk
32
33            # If EOF, this boundary should be at the end of the
file
34            if mini_chunk == b"":
35                chunk_boundaries[bi] = file_size
36                break
37
38            # Find the special token in the mini chunk
39            found_at = mini_chunk.find(split_special_token)
40            if found_at != -1:
41                chunk_boundaries[bi] = initial_position +
found_at
42                break
43            initial_position += mini_chunk_size
44

```



```

45     # Make sure all boundaries are unique, but might be fewer
    than desired_num_chunks
46     return sorted(set(chunk_boundaries))
47
48
49 def get_new_split(old_split: list[bytes], pair: tuple[bytes,
bytes], merged: bytes) -> list[bytes]:
50     new_split = []
51     i = 0
52     while i < len(old_split):
53         if i < len(old_split) - 1 and (old_split[i], old_split[i
+ 1]) == pair:
54             new_split.append(merged)
55             i += 2
56         else:
57             new_split.append(old_split[i])
58             i += 1
59     return new_split
60 def get_bpe_train(
61     input_path: str | os.PathLike,
62     vocab_size: int,
63     special_tokens: list[str],
64     **kwargs,
65 ) -> tuple[dict[int, bytes], list[tuple[bytes, bytes]]]:
66     PAT = r"('(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?
    [\^s\p{L}\p{N}]+\s+(?!S)|\s+)"
67
68     # 初始化vocab
69     # 256个单字节 + special tokens
70     # 注意是bytes([i]), 传入一个可迭代对象, 否则会产生i个0
71     vocab: dict[int, bytes] = {i: bytes([i]) for i in range(256)}
72     # 导入特殊符号
73     special_tokens_vocab = {256 + i: token.encode("utf-8") for i,
token in enumerate(special_tokens)}
74     # 合并
75     vocab.update(special_tokens_vocab)
76
77     # 编译pre_token过程中要使用的正则表达式
78     pre_token_re = re.compile(PAT)
79
80     # 编译分离特殊符号的正则表达式
81     special_pat = "|".join(re.escape(t) for t in special_tokens)
    # 注意转义, 最终是形如 a|b|c 的形式
82     special_re = re.compile(special_pat) if special_pat != ""
    else None

```

```

83 pair_dict = PairDict()
84 token_dict = TokenDict()
85 merges: list[tuple[bytes, bytes]] = []
86
87 # 初始化token_dict和pair_dict
88 with open(input_path, "rb") as f:
89     num_chunks = os.cpu_count()
90     boundaries = find_chunk_boundaries(f, num_chunks, b"
<|endoftext|>")
91     for start, end in zip(boundaries[:-1], boundaries[1:]):
92         f.seek(start)
93         # 注意此时chunk由bytes类型转换为str类型，可以应用正则表达式
94         chunk = f.read(end - start).decode("utf-8",
errors="ignore")
95         # 接下来使用正则表达式进行一些处理
96         # 首先，需要调整换行符号，原因在报错中可以发现
97         chunk = re.sub(r"\r\n?", "\n", chunk)
98         # 其次，去除其中的特殊符号，根据特殊符号进行分段
99         if special_re is not None:
100             segments = special_re.split(chunk)
101         else:
102             segments = [chunk]
103         # 最后，对每一个segment进行预分词
104         for seg in segments:
105             if not seg:
106                 continue
107             for token_str_match in
pre_token_re.finditer(seg):
108                 token_str = token_str_match.group(0)
109                 token_by = token_str.encode("utf-8") # 转为
bytes类型
110
111                 # 更新token_dict，将token_by加入token_dict，频次
计数加一，划分模式为单个字符
112                 try:
113                     token_dict.token2count[token_by] += 1
114                 except KeyError:
115                     token_split = [bytes([b]) for b in
token_by]
116                     token_dict.token2count[token_by] = 1
117                     token_dict.token2splits[token_by] =
token_split
118
119                 # 更新pair_dict，统计token_by中相邻字符对的频次

```

```

121         token_split =
token_dict.token2splits[token_by]
122         for adj_pair in zip(token_split[:-1],
token_split[1:]):
123             pair_dict.add_pair(adj_pair, 1, token_by)
124
125     while len(vocab) < vocab_size:
126         max_pair = pair_dict.get_max_pair()
127         merged_pair = max_pair[0] + max_pair[1]
128         merges.append(max_pair)
129         vocab[len(vocab)] = merged_pair
130         # 对所有包含该pair的token进行更新
131         # 首先根据pair_dict找到包含该pair的token
132         # 注意这里使用pop而不是直接访问，是因为后续不再需要访问该pair了
133         # 但是由于key同时会被pop，因此后续访问时可能会出现KeyError
134         tokens_to_update = pair_dict.pair2tokens.pop(max_pair)
135         # 最后，统计每个token中该pair的出现次数，注意根据划分模式进行统计
136         for token in tokens_to_update:
137             token_freq = token_dict.token2count[token]
138             old_split = token_dict.token2splits[token]
139             new_split = get_new_split(old_split, max_pair,
merged_pair)
140             # 统计旧划分模式中该pair的出现次数
141             old_adj_pairs = list(zip(old_split[:-1],
old_split[1:]))
142             new_adj_pairs = list(zip(new_split[:-1],
new_split[1:]))
143             # 更新token_dict中的划分方式
144             token_dict.token2splits[token] = new_split
145
146             old_cnt = Counter(old_adj_pairs)
147             new_cnt = Counter(new_adj_pairs)
148
149             # 更新计数
150             for pair, count in old_cnt.items():
151                 # count是pair在该token中出现的次数，token_freq是该
token在语料库中出现的次数
152                 # 二者的乘积就是该pair在语料库中出现的次数
153                 # 更新非合并对的频数和位置
154                 try:
155                     # 这里可能会出现KeyError，因为某些pair可能已经被合并
掉了
156                     pair_dict.discard_pair(pair, count *
token_freq, token)
157                 except KeyError:

```

```

158                                     # 已经作为max_pair被合并掉了
159                                     # pair2tokens会有KeyError
160                                     continue
161
162         for pair, count in new_cnt.items():
163             # 与上述类似
164
165             pair_dict.add_pair(pair, count * token_freq,
166                                token)
167
167     return vocab, merges

```

BPE Tokenizer的实现 (2) - Encode-Decode部分

原理

实现一个对象tokenizer对象，能够从文件读入json格式的vocab和txt格式的merges列表

读入过程很简单，关键在于读入后需要初始化一些构造，例如id到bytes和bytes到id的映射，vocab中所有bytes的集合方便速查，merges中的bytes-pair到顺序的映射方便快速查找合并对象

具体实现过程中

- decode很简单，输入一个id序列，根据id到bytes映射依次翻译过去即可
- encode的逻辑主要在于选择合适的id，事实上每一个token都可能具有多种划分方法，一个最简单的例子是，假如vocab中存在“ab”和“bc”，那么对于token：“abc”，到底是应该划分为“a | bc”还是“ab | c”，这里影响选择的的就是这两组bytes-pair在merges列表中的相对顺序，根据原文档的描述，靠前的bytes-pair应该被优先选中。在实现时，需要维护一个循环来划分token，初始为单字母划分，每次从现在的划分方法中找到一个序号最小的候选pair，之后使用该pair更新划分方法，直到该token中找不到新的pair为止

思考

与上述相似，主要在于运行过程能描述清楚

实现

需要额外描述的东西不多，只有极个别要点

- 可以将encode中最前面那一些正则表达式转移到初始化过程中，能提升一点性能
- 不管有没有特殊符号，都要进行分词，如果不分词就是对整个text进行encode过程，

虽然最终结果也是对的，但是每次都要遍历全部文本进行划分的查找与更新，非常浪费性能，实测运行时长能达到预分词的二到三倍

- iterable方法需要使用yield关键字，用于每次调用的时候自动向下迭代一次，一般是迭代器会使用的
- decode过程对无法识别的id使用replace模式，这个是原文档中要求的，但是不清楚测试中是否有包含在内？

完整代码

```
1 class BPETokenizer:
2     def __init__(
3         self,
4         vocab: dict[int, bytes] = None,
5         merges: list[tuple[bytes, bytes]] = None,
6         special_tokens: list[str] | None = None):
7         self.vocab = vocab
8         self.merges = merges
9         self.special_tokens = special_tokens
10        self.bytes2idx = {v: k for k, v in vocab.items()} if
vocab is not None else {}
11        self.bytes_set = set(self.bytes2idx.keys())
12        self.mergepair2idx = {pair: i for i, pair in
enumerate(merges)} if merges is not None else {}
13        self.max_token_length = max(len(v) for v in
self.bytes2idx.keys()) if vocab is not None else 0
14
15
16        @classmethod
17        def from_files(cls, vocab_filepath: str, merges_filepath:
str, special_tokens: list[str] | None = None):
18            # 观察fixtures中的文件可以发现，vocab文件保存为json，merges文件保
存为txt，每一行是一个pair，由空格分隔
19            # 导入vocab
20            import json
21            with open(vocab_filepath, "r", encoding="utf-8") as f:
22                vocab = json.load(f)
23                # 将key从str转换为int
24                vocab = {int(k): v.encode("utf-8") for k, v in
vocab.items()}
25            # 导入merges
26            merges = []
27            with open(merges_filepath, "r", encoding="utf-8") as f:
28                for line in f:
29                    parts = line.strip().split()
```

```

30         if len(parts) != 2:
31             continue
32         merges.append((parts[0].encode("utf-8"),
parts[1].encode("utf-8")))
33     return cls(vocab, merges, special_tokens)
34
35     def encode(self, text: str) -> list[int]:
36         PAT = r"''('?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?
37         [^\s\p{L}\p{N}]+\s+(?!\\S)|\\s+''"
38         idxs = []
39         # 将special_tokens按长度从大到小排序，避免子串被优先匹配
40         special_tokens_sorted = sorted(self.special_tokens or [],
key=len, reverse=True)
41         special_pat = "(?:" + "|".join(
42             re.escape(t) for t in special_tokens_sorted) + ")" if
special_tokens_sorted else ""
43
44         pre_token_re = re.compile(PAT)
45         special_token_by_set:set[bytes] = set(t.encode("utf-8")
for t in special_tokens_sorted)
46
47         # 首先根据特殊符号进行分段，将每一段进行预分词，然后将特殊符号和预分词
结果交替合并
48         if special_tokens_sorted:
49             special_re = re.compile(special_pat)
50             segments = special_re.split(text)
51             specials = special_re.findall(text)
52             # 交替合并segments和specials
53             combined:list[bytes] = []
54             for seg, spec in zip(segments, specials + [""]):
55                 # 如果第一个token就是special token，那么seg会是空字符串
56                 # 所以可以直接if seg，避免加入空字符串
57                 if seg:
58                     # 使用预分词正则表达式进行预分词，将分词结果保存下来
59                     tokens = pre_token_re.findall(seg)
60                     for token in tokens:
61                         combined.append(token.encode("utf-8"))
62                 if spec:
63                     combined.append(spec.encode("utf-8"))
64             else:
65                 combined = [ token.encode("utf-8") for token in
pre_token_re.findall(text)]
66
67         # 到这里为止，combined中已经包含了所有的预分词结果和特殊符号
68         # 此时combined中每个元素要么是特殊符号，要么是完成预分词的token
69         # 准备开始编码

```

```

68         tokens_by : list[bytes] = []
69         for token in combined:
70             if token in special_token_by_set:
71                 tokens_by.append(token)
72                 continue
73             token_split: list[bytes] = [bytes([b]) for b in token]
74
75             while True:
76                 # 在当前划分模式下，找出所有相邻对中出现于mergepair2idx中
77                 # 的pair
78                 candidate_pairs = [(pair,
79 self.mergepair2idx[pair]) for pair in zip(token_split[:-1],
80 token_split[1:])] if pair in self.mergepair2idx]
81                 if not candidate_pairs:
82                     break
83                 # 选择其中index最小的pair进行合并
84                 best_pair = min(candidate_pairs, key=lambda x:
85 x[1])[0]
86                 token_split = get_new_split(token_split,
87 best_pair, best_pair[0] + best_pair[1])
88                 tokens_by.extend(token_split)
89                 # 最后，将tokens_by转换为对应的idx
90                 for token in tokens_by:
91                     if token in self.bytes2idx:
92                         idxs.append(self.bytes2idx[token])
93                     else:
94                         raise ValueError(f"Token {token} not in
95 vocabulary.")
96
97         return idxs
98
99     def encode_iterable(self, iterable: Iterable[str]) ->
100 Iterator[int]:
101         """
102         Given an iterable of strings (e.g., a Python file
103         handle), return a generator that lazily yields token IDs.
104         This is required for memory-efficient tokenization of
105         large files that we cannot directly load into memory.
106         """
107         for text in iterable:

```

```

104         for idx in self.encode(text):
105             yield idx
106
107
108     def decode(self, ids: list[int]) -> str:
109         bytes_list = [self.vocab[i] for i in ids]
110         return b"".join(bytes_list).decode("utf-8",
errors="replace")

```

Transformer的实现

原理

主要是根据原文档中指出的相关矩阵运算进行编写，最好使用einsum库来显式标识矩阵维度的含义

左图是Transformer模型整体架构，右图是一个Transformer块的组成

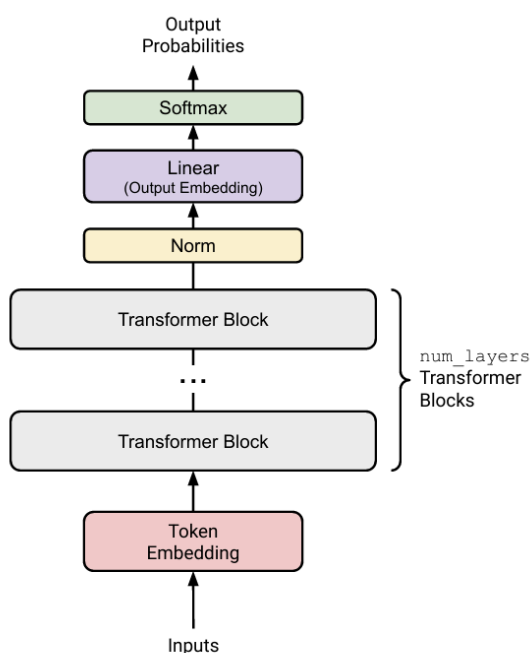


Figure 1: An overview of our Transformer language model.

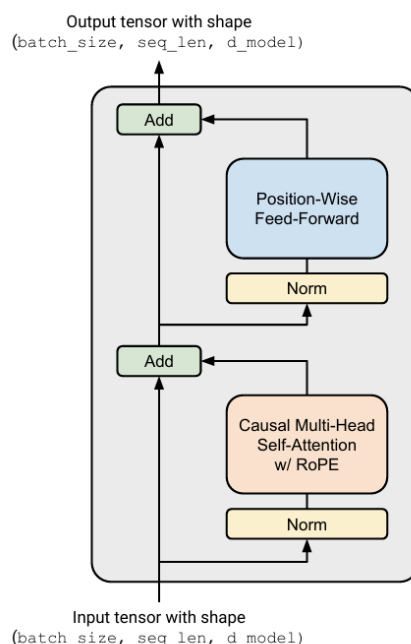


Figure 2: A pre-norm Transformer block.

本节依次实现

- 线性层Linear
- 嵌入层Embedding
- 归一化层RMSNorm
- 激活函数SWIGLU（在Position-Wise-Feed-Forward中）

- 旋转位置编码RoPE
- 激活函数softmax
- 多头注意力MHA
- 组装Transformer Block
- 组装Transformer模型

思考

本节主要复述一下相关原理和公式，方便手撕的时候背诵，其中rope，MHA等代码实现都是手撕的重点

构思一下数据在transformer中经历了什么

- 输入(batch_size, sequence_length)
- 嵌入(batch_size, sequence_length, d_model)
- pre-norm过程(batch_size, sequence_length, d_model)
- 注意力计算(batch_size, sequence_length, d_model)

实现

Linear模块

- 输入(... , in_features)
- 权重(out_features , in_features)
- 输出(... , out_features)
- 初始化权重

- - Linear weights: $\mathcal{N}\left(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}}\right)$ truncated at $[-3\sigma, 3\sigma]$.
 - Embedding: $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ truncated at $[-3, 3]$
 - RMSNorm: $\mathbf{1}$

```

○ 1  self.weight =
    nn.Parameter(torch.empty((out_features, in_features), device
    =device, dtype=dtype))
2      # self.bias =
    nn.Parameter(torch.empty((out_features,), device=device, dtype=
    dtype))
3      # 使用trunc_normal_初始化权重
4      # 原地初始化, 避免重绑定导致丢失 Parameter 身份
5      mu = 0.0
6      std = math.sqrt(2 / (in_features + out_features))
7      a = -3 * std
8      b = 3 * std
9      nn.init.trunc_normal_(self.weight, mean=mu, std=std,
    a=a, b=b)

```

- 实现forward方法, 由于输入的最后一维参加运算, 所以输入x的形状是(... , in_features), 为了保证这是一个线性变换, 所以输出y的形状是(... , out_features), 再根据原文档的要求, 实质上实现的是 $y = xW^T$, W不允许保存为转置, 所以权重的形状是(out_features , in_features)
- 使用einsum描述方法

```

1  return einsum(
2      x, self.weight,
3      "... in_features, out_features in_features -> ...
    out_features"
4  )

```

Embedding模块

- 输入(batch_size, sequence_length)
- 权重(vocab_size, d_model)
- 输出(batch_size, sequence_length, d_model)
- 将seq_len维的每一个id都映射为一个大小为d_model的向量, 所以本模块不是矩阵乘法操作, 而是访问操作

```

○ 1  def forward(self, token_ids: torch.Tensor) ->
    torch.Tensor:
2      return self.weight[token_ids]

```

- 初始化权重

- ```
1 self.weight =
 nn.Parameter(torch.empty((num_embeddings, embedding_dim), device=device, dtype=dtype))
2 nn.init.trunc_normal_(self.weight, mean=0.0, std=1.0, a=-3.0, b=3.0)
```

- 注意num\_embeddings 就是 vocab\_size; embedding\_dim 就是 d\_model

## RMSNorm模块

- 原理

(RMSNorm; [Zhang and Sennrich, 2019](#), equation 4) for layer normalization. Given a vector  $a \in \mathbb{R}^{d_{\text{model}}}$  of activations, RMSNorm will rescale each activation  $a_i$  as follows:

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

where  $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$ . Here,  $g_i$  is a learnable “gain” parameter (there are  $d_{\text{model}}$  such parameters total), and  $\varepsilon$  is a hyperparameter that is often fixed at  $1e-5$ .

- 注意在这里输入的最后二维需要调整类型为float32，并在最后调整回原本的数据类型，猜测可能是因为会有半精度的数据输入
- 先根据最后一维计算RMS，再依次进行归一化，注意模块存在一个可学习的参数，形状是(d\_model)，刚好和输入的最后二维一一对应，注意要初始化为全1，不能是全0或者正态分布

- ```
1 def forward(self, x: torch.Tensor) -> torch.Tensor:
2     in_dtype = x.dtype
3     x = x.to(torch.float32)
4
5     rms = self.RMS(x)
6     x_norm = x / rms
7     result = x_norm * self.weight
8     # Return the result in the original dtype
9     return result.to(in_dtype)
10
11 def RMS(self, x: torch.Tensor) -> torch.Tensor:
12     assert x.shape[-1] == self.d_model
13     return torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
```

SwiGLU

- 是一种SiLU和GLU的结合



- SiLU的原理
$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

- GLU的原理
$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x,$$

- SwiGLU的原理

Putting the SiLU/Swish and GLU together, we get the SwiGLU, which we will use for our feed-forward networks:

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

where $x \in \mathbb{R}^{d_{\text{model}}}$, $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and canonically, $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$.

- 注意，上述所有形如 Wx 的矩阵运算，在实现的时候都是 xW^T
- 事先声明好过程中用到的权重矩阵，在FFN层中的SwiGLU，会声明一个 d_{ff} 维度，大小大概是模型维度的8/3，注意三个权重矩阵的形状分别是 $(d_{\text{ff}}, d_{\text{model}})$ 、 $(d_{\text{model}}, d_{\text{ff}})$ 、 $(d_{\text{ff}}, d_{\text{model}})$
- 再次强调，为了方便计算机存储，所有一维向量都是行向量，虽然它们在数学上可能是一个列向量；与此同时，权重矩阵仍然存储为实现数学上的运算“ Wx ”中的权重矩阵，也就是说，此时数学上的 x 的形状是 $(d_{\text{model}}, 1)$ ，故有 W 的形状是 $(d_{\text{ff}}, d_{\text{model}})$

- ```
1 self.w1 = Linear(d_ff, d_model, device=device, dtype=dtype)
2 # w1: d_ff, d_model
3 self.w2 = Linear(d_model, d_ff, device=device, dtype=dtype)
4 # w2: d_model, d_ff
5 self.w3 = Linear(d_ff, d_model, device=device, dtype=dtype)
```

- 在forward方法中直接按照公式实现即可

- ```
1 wx1 = self.w1(in_feature) # ... d_ff
2 silu = wx1 * torch.sigmoid(wx1)
3 wx3 = self.w3(in_feature) # ... d_ff
4 x2 = silu * wx3 # ... d_ff
5 wx2 = self.w2(x2) # ... d_model
6 return wx2
```

RoPE

- 很神奇的一种编码方式，是一种绝对位置编码，但是具有良好的外推性，具有一定的相对位置编码的特性

以下笔记将大量参考网络内容

- 如果仅使用qkv相乘的做法，将会使transformer丢失相对于LSTM的一些能力，例如序列的时序性，或者说是不同token之间的相对性，因此需要对向量进行位置编码，将位置信息赋予词向量，从而保证该能力
- 形如 $q_m = f_q(x_m, m), k_n = f_k(x_n, n), v_n = f_v(x_n, n)$ 所谓的编码方式就是找到一个合适的 f_q, f_k, f_v

- $$a_{m,n} = \frac{\exp(\frac{q_m^T k_n}{\sqrt{d}})}{\sum_{j=1}^N \exp(\frac{q_m^T k_j}{\sqrt{d}})}, o_m = \sum_{n=1}^N a_{m,n} v_n$$

- 绝对位置编码

- $f_{q,k,v}(x_i, i) = W_{q,k,v}(x_i + p_i)$ 其中 x_i 和 p_i 都是一个d维的向量
- 对于 p_i 向量中每一个位置的元素的计算方法，采用正余弦交替取值
- $p_{i,2t} = \sin(\frac{i}{10000^{\frac{2t}{d}}})$
- $p_{i,2t+1} = \cos(\frac{i}{10000^{\frac{2t}{d}}})$
- 注意其中10000是一个超参数，其指数均为 $\frac{2t}{d}$

- 为了能够利用相对位置信息，提出存在一个函数g有如下性质

- 内积 $\langle f_q(x_m, m), f_k(x_n, n) \rangle = g(x_m, x_n, m - n)$
- 对于二维情况下的向量，有

$$f_q(\mathbf{x}_m, m) = (W_q \mathbf{x}_m) e^{im\theta}$$

$$f_k(\mathbf{x}_n, n) = (W_k \mathbf{x}_n) e^{in\theta}$$

$$g(\mathbf{x}_m, \mathbf{x}_n, m - n) = \text{Re} \left[(W_q \mathbf{x}_m) (W_k \mathbf{x}_n)^* e^{i(m-n)\theta} \right]$$

- 将 $e^{im\theta}$ 展开，就有旋转矩阵

$$\begin{aligned} f_q(\mathbf{x}_m, m) &= \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_q^{(1,1)} & W_q^{(1,2)} \\ W_q^{(2,1)} & W_q^{(2,2)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \\ &= \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix} \end{aligned}$$

- 对于旋转矩阵 Ra ，有性质 $(Ra)^T = R(-a)$; $RaRb = R(a + b)$

◦ 于是 $f_q(x_m, m) = R(m\theta)q_m$

◦ 则内积

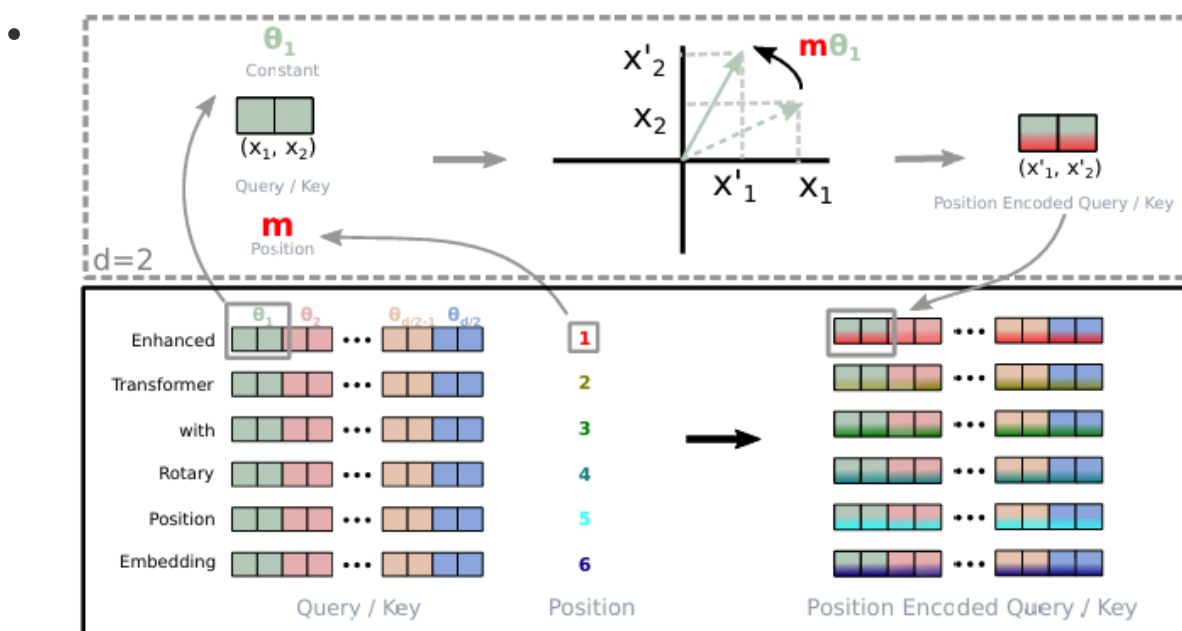
$$\langle f_q, f_k \rangle = (R(m\theta)q_m)^T R(n\theta)k_n = q_m^T R(-m\theta)R(n\theta)k_n = q_m^T R((n-m)\theta)k_n$$

- 当扩展到多维情况下时，可以将向量每两个元素一组，仍然可以实现旋转位置编码
- 内积满足线性叠加性，因此任意偶数维的RoPE，我们都可以表示为二维情形的拼接，即

$$R_{\Theta, m}^d = \underbrace{\begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{pmatrix}}_{W_m} \quad (13)$$

$$\Theta = \left\{ \theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2] \right\}$$

- 此外，由于R矩阵是一个正交矩阵，正交矩阵不会改变向量的模长，因此也通常不会影响模型的稳定性



- 代码实现的思路

◦ 首先需要预计算R矩阵，注意R矩阵本身就十分稀疏，没必要完整计算 $d_k \times d_k$ ，另一方面，其实 $\max_seq_len$ 未必就小于 d_k ，为了良好的外推性，R矩阵最好是 $\max_seq_len \times d_k$ 的

◦ AI一下

◦ ■ R 不是一个整体的 $d_k \times d_k$ 矩阵

因为：

■ 对于每个位置 p ，整个 d_k -维旋转操作是由 $d_k/2$ 个独立的 2×2

旋转矩阵块组成；

- 这些 2×2 矩阵是按维度分块独立作用的，而不是全维度混合变换；
- 因此，整个旋转矩阵 R_p 实际上是一个 **block-diagonal** 矩阵：

$$R_p = \begin{bmatrix} R_{p,0} & 0 & \cdots & 0 \\ 0 & R_{p,1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & R_{p,d_k/2-1} \end{bmatrix}$$

其中每个 $R_{p,i}$ 是 2×2 。

所以如果你真要展开成一个完整矩阵，它确实是 $d_k \times d_k$ ，但那是一个非常稀疏的块对角矩阵，没必要显式构造。

- 为什么代码里是 `(max_seq_len, d_k)` 的矩阵？

因为：

1. 对于每个位置 p ，我们只需要存储对应每个维度的旋转角度（正弦和余弦）；
2. 每两个维度共享同一个角度参数；
3. 因此可以存成两个矩阵：
 - `cosine: (max_seq_len, d_k/2)`
 - `sine: (max_seq_len, d_k/2)`

在 forward 时，代码按如下方式对齐：

```
1 | x_even * cos - x_odd * sin
2 | x_even * sin + x_odd * cos
```

也就是在每个位置、每对维度上应用旋转。

- 为什么不是 `(max_seq_len × max_seq_len)`？

因为：

- RoPE 不涉及 **token-to-token** 的交互（不像 self-attention 那样有序列间耦合）；
- 它只对每个 token 的自身向量施加旋转；
- 因此，与位置 p 相关的旋转仅仅依赖于 p 和维度索引 i ，而不是两个 token 的位置 p_1, p_2 ；
- 所以我们不需要一个 `max_seq_len × max_seq_len` 的二维关

系矩阵。

- 在完成预计算后，需要缓存在类中，方便之后多次使用
- 剩下的步骤就很简单了，根据两个一组的原则，将输入的最后一维按照奇偶分开，依次从偶数组、奇数组各抽出一个（注意，旋转的是词向量各个元素，不能理解为token之间的旋转）
- 将计算结果在最后一维stack，其大小为2，再通过rearrange合并最后一维，这种合并方法会从最后一维的两个元素中依次取数，从而实现结果的奇偶合并

- ```
1 class RoPE(nn.Module):
2 def __init__(self, theta: float, d_k: int, max_seq_len:
3 int, device: torch.device | None = None):
4 super().__init__()
5 assert d_k % 2 == 0, "d_k must be even"
6 self.theta = theta
7 self.d_k = d_k
8
9 self.max_seq_len = max_seq_len
10 self.device = device
11
12 i = rearrange(torch.arange(max_seq_len), "n -> n 1")
13 k = rearrange(torch.arange(d_k//2), "d_kdiv2 -> 1
14 d_kdiv2")
15 angle_rates = 1 / (theta ** (2 * k / d_k)) # (1,
16 d_k/2)
17 angle = i * angle_rates # (max_seq_len, d_k/2)
18 self.cosine = torch.cos(angle).to(device) #
19 (max_seq_len, d_k/2)
20 self.sine = torch.sin(angle).to(device) #
21 (max_seq_len, d_k/2)
22 self.register_buffer("cosine_buffer", self.cosine)
23 self.register_buffer("sine_buffer", self.sine)
24
25 def forward(self, x: torch.Tensor, token_positions:
26 torch.Tensor)-> torch.Tensor:
27 token_positions = token_positions.to(torch.int)
28 cos = self.cosine_buffer[token_positions] #
29 (batch_size, seq_len, d_k/2)
30 sin = self.sine_buffer[token_positions] # (batch_size,
31 seq_len, d_k/2)
32 x_even = x[..., ::2] # (batch_size, seq_len, d_k/2)
33 x_odd = x[..., 1::2] # (batch_size, seq_len, d_k/2)
34 x_rotated_even = x_even * cos - x_odd * sin #
35 (batch_size, seq_len, d_k/2)
```



```

27 x_rotated_odd = x_even * sin + x_odd * cos #
 (batch_size, seq_len, d_k/2)
28 x_rotated = torch.stack((x_rotated_even,
 x_rotated_odd), dim=-1) # (batch_size, seq_len, d_k/2, 2)
29 x_rotated = rearrange(x_rotated, "... d_kdiv2 two ->
 ... (d_kdiv2 two)")
30 return x_rotated

```

## softmax

- 其实没什么太多需要注意的，原理如下
- 

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}.$$

- 注意softmax操作中，所有数加减一个常数不影响结果，因此可以所有数减去最大值，从而提高稳定性

- ```

1 def softmax(x:torch.Tensor,dim:int)->torch.Tensor:
2     x_m = torch.max(x,dim=dim,keepdim=True).values
3     x_exp = torch.exp(x - x_m)
4     x_exp_sum = x_exp.sum(dim=dim,keepdim=True)
5     return x_exp / x_exp_sum

```

MHA

- 实现多头注意力之前，首先要实现scaled_dot_product_attention模块来计算注意力分数
 - 这其实就是我们一般意义上的注意力计算，接受QKV和mask矩阵作为参数
 - 公式如下

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q^T K}{\sqrt{d_k}} \right) V$$

- 当然，仍然要注意，这里公式的描述都是按照数学形式来的，实际存储的是行列交换的，因此计算时需要注意是否转置
- Q的形状是(..., query, d_k), K的形状是(..., key, d_k), 因此使用的einsum描述是"... queries d_k, ... keys d_k -> ... queries keys", 计算得到注意力分数

- 注意需要使用mask矩阵进行遮盖，通过对注意力分数使用masked_fill方法，根据mask矩阵中标记为False的位置将注意力分数对应全部填补为-inf，即可实现mask功能
- 为什么注意力分数要除以根号d
 - 点积的数量级增长很大，因此将 softmax 函数推向了梯度极小的区域
 - 假设q,v各个元素是均值为0，方差为1的正态分布，有两个向量的内积的方差为维度d，则除以 $\sqrt{d}$ 可以使得注意力分数的方差仍然为1

```

1  def
   scaled_dot_product_attention(Q:torch.Tensor,K:torch.Tenso
   r,V:torch.Tensor,mask:torch.Tensor):
2      d_k = Q.shape[-1]
3      scores = einsum(
4          Q , K,
5          "... queries d_k , ... keys d_k -> ... queries
   keys"
6      )
7      scores = scores /
   torch.sqrt(torch.tensor(d_k,dtype=scores.dtype))
8      for i in range(len(mask.shape)-len(scores.shape)):
9          mask = mask.unsqueeze(0)
10         # 注意masked_fill_的逻辑和原mask矩阵不同
11         # 需要将mask中为False的位置填充为-inf
12         scores = scores.masked_fill(mask==False,float("-
   inf"))
13         attn = softmax(scores,dim=-1)
14         output = einsum(
15             attn,V,
16             "... queries keys, ... keys d_v -> ... queries
   d_v"
17         )
18         return output

```

- 多头注意力实际上就是将最后一个维度分给了多个注意力头来计算，此时每个头维护的维度 $d_k = d_{model} / num_{heads}$ ，其中 d_{model} 就是整个模型词向量的维度，也就是在MHA之前的模块中所说的 d_K ，需要注意区分
- 计算多头注意力的公式如下所示

- $$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

with Q_i, K_i, V_i being slice number $i \in \{1, \dots, h\}$ of size d_k or d_v of the embedding dimension for Q, K , and V respectively. With Attention being the scaled dot-product attention operation defined in §3.5.4. From this we can form the multi-head *self*-attention operation:

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

Here, the learnable parameters are $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$, $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$, and $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$. Since the Q s, K s, and V s are sliced in the multi-head attention operation, we can think of W_Q , W_K and W_V as being separated for each head along the output dimension. When you have this working, you should be computing the key, value, and query projections in a total of three matrix multiplies.

- 在实际实现时，没有必要为每一个头都重新维护QKV的权重矩阵，事实上，这些每个头的权重矩阵本来就应该是一个权重矩阵
- 因此，可以直接将权重矩阵声明为形状为(d_model, d_model)的方阵，在依次计算每个头的注意力的时候，使用einsum，调整其维度"batch_size seq_len (num_heads d_k) -> batch_size num_heads seq_len d_k"
- 注意需要将num_heads维提到前面，这样才能满足依次计算每个样本 -> 计算每个头的计算顺序
- 在计算时，注意MHA采用Causal masking，也就是依次遮蔽下一个token，使得模型学习并预测下一个token，在实现时，即生成一个下三角包含主对角线为1的矩阵作为mask矩阵即可
 - 注意mask是相对于token来说的，rope是相对词向量元素来说的；换成好理解的话，就是mask每次遮蔽了当前词向量的下一个词向量，在表现上就是将注意力分数置为-inf，从而使得计算softmax时，该项失去意义 ($e^{(-\text{inf})} = 0$)，而rope是对一个词向量内部的元素，结合该词向量的相对位置进行旋转，从而将位置信息嵌入到词向量中
- 之后对输入的q, k进行rope编码，完成后即可进行注意力的计算，完成计算后使用einsum将多头注意力合并"batch_size num_heads seq_len d_v -> batch_size seq_len (num_heads d_v)"
- 最后结果需要通过输出的权重矩阵，形状是(d_model, d_model)
- 完整代码如下

- ```

1 class MultiheadSelfAttention(nn.Module):
2 def __init__(self,
3 d_model:int,
4 num_heads:int,
5 theta:float = None,
6 max_seq_len:int=None,
7 token_positions=None,
8 device:torch.device|None=None,
9 dtype:torch.dtype|None=None):
10 super().__init__()

```

```

11 assert d_model % num_heads == 0, "d_model must be
divisible by num_heads"
12 self.d_model = d_model
13 self.num_heads = num_heads
14 self.d_k = d_model // num_heads
15 self.d_v = self.d_k
16 self.wq =
Linear(d_model, d_model, device=device, dtype=dtype)
17 self.wk =
Linear(d_model, d_model, device=device, dtype=dtype)
18 self.wv =
Linear(d_model, d_model, device=device, dtype=dtype)
19 self.wo =
Linear(d_model, d_model, device=device, dtype=dtype)
20
21
22 if theta is not None and max_seq_len is not None:
23 self.rope =
RoPE(theta, d_model//num_heads, max_seq_len, device=device)
24 self.token_positions = token_positions
25
26
27 def forward(self, Q, K, V):
28
29 seq_len = Q.shape[1]
30 xq, xk, xv = self.wq(Q), self.wk(K), self.wv(V) #
(batch_size, seq_len, d_model)
31 xq = rearrange(
32 xq,
33 "batch_size seq_len (num_heads d_k) -> batch_size
num_heads seq_len d_k",
34 num_heads = self.num_heads,
35 d_k = self.d_k,
36)
37 xk = rearrange(
38 xk,
39 "batch_size seq_len (num_heads d_k) -> batch_size
num_heads seq_len d_k",
40 num_heads = self.num_heads,
41 d_k = self.d_k,
42)
43 xv = rearrange(
44 xv,
45 "batch_size seq_len (num_heads d_k) -> batch_size
num_heads seq_len d_k",

```

```

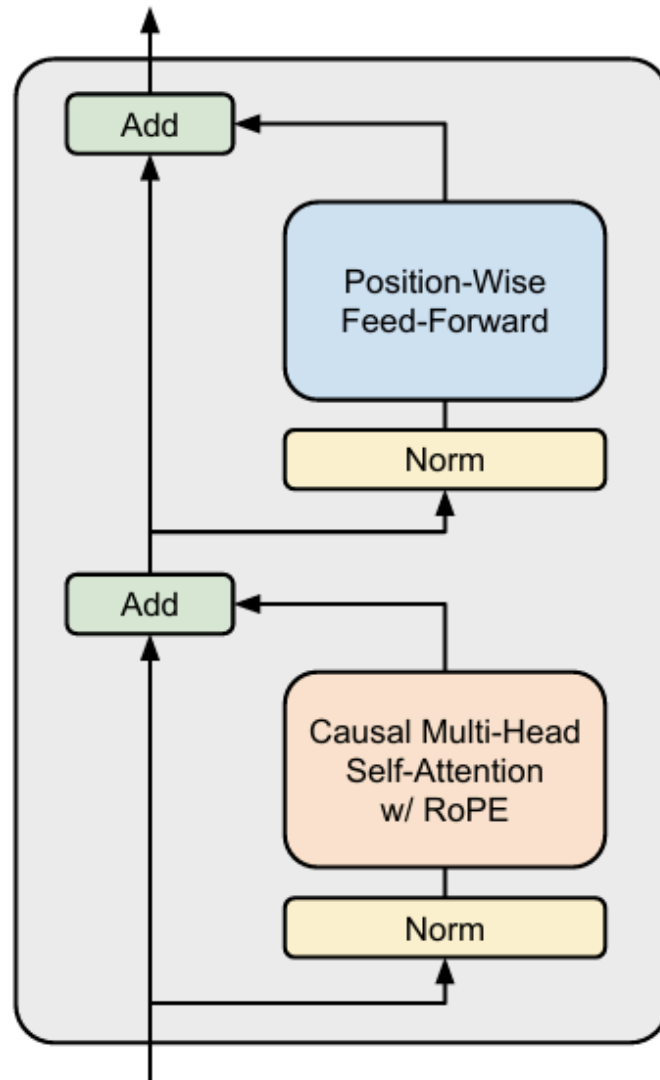
46 num_heads = self.num_heads,
47 d_k = self.d_k,
48)
49 # 保留一个下三角为1，其余为0的掩码矩阵，默认包括主对角线
50 attention_mask =
torch.tril(torch.ones((seq_len, seq_len), dtype=torch.bool, device=Q.device))
51 if hasattr(self, "rope"):
52 token_positions = self.token_positions
53 xq = self.rope(xq, token_positions)
54 xk = self.rope(xk, token_positions)
55 x =
scaled_dot_product_attention(xq, xk, xv, attention_mask) #
(batch_size, num_heads, seq_len, d_v)
56 x = rearrange(
57 x,
58 "batch_size num_heads seq_len d_v -> batch_size
seq_len (num_heads d_v)"
59) # (batch_size, seq_len, d_model)
60 x = self.wo(x) # (batch_size, seq_len, d_model)
61 return x

```

## Transformer Block

- 一个Transformer Block内部有两个norm层，一个因果多头注意力模块，一个FFN模块，每个模块都是pre-norm的，且都使用残差连接
- 原理图如下

Output tensor with shape  
(batch\_size, seq\_len, d\_model)



Input tensor with shape  
(batch\_size, seq\_len, d\_model)

- 按照示意图组装上述模块即可，注意需要手动为MHA模块生成token\_positions
- 除此之外，注意使用残差连接，以自注意力这里为例，ADD实际上就是将自注意力输出和输入相加
- 注意：如果你对为什么token\_position在这里生成有疑问的话，请参见 `adapters.py` 中所使用的测试接口，测试的时候会有是否使用rope的区别，如果使用rope，会在外部传入一个positon，这个positon不能保证一定是从零开始的，因此在内部需要暴露一个接口来接受position传入。但是在整个block的层面来看，输入一定是完整的，因此没有必要接受positon，也没有一个模块有资格传入positon来控制训练过程。因此，positon需要在block层面进行初始化，**如果不在这里初始化，说明一定只是在测试功能**

```
1 class TransformerBlock(nn.Module):
```

```

2 def __init__(self,
3 d_model:int,
4 num_heads:int,
5 d_ff:int,
6 max_seq_len:int,
7 theta:float,
8 device:torch.device|None=None,
9 dtype:torch.dtype|None=None
10):
11 super().__init__()
12 self.d_model = d_model
13 self.num_heads = num_heads
14 self.d_ff = d_ff
15 self.max_seq_len = max_seq_len
16 self.theta = theta
17 self.lm1 =
18 RMSNorm(self.d_model, device=device, dtype=dtype)
19 self.lm2 =
20 RMSNorm(self.d_model, device=device, dtype=dtype)
21 self.ffn =
22 SwiGLU(self.d_model, self.d_ff, device=device, dtype=dtype)
23
24 self.mha = MultiheadSelfAttention(
25 self.d_model,
26 self.num_heads,
27 self.theta,
28 self.max_seq_len,
29 device=device,
30 dtype=dtype
31)
32
33 def forward(self, x:torch.Tensor):
34 # x (batch_size, seq_len, d_model)
35 batch_size = x.shape[0]
36 seq_len = x.shape[1]
37 assert self.d_model == x.shape[2]
38
39 token_positons =
40 torch.arange(seq_len, device=x.device, dtype=torch.int)
41 self.mha.token_positions = token_positons
42
43 y = x + self.mha(self.lm1(x), self.lm1(x), self.lm1(x))

```

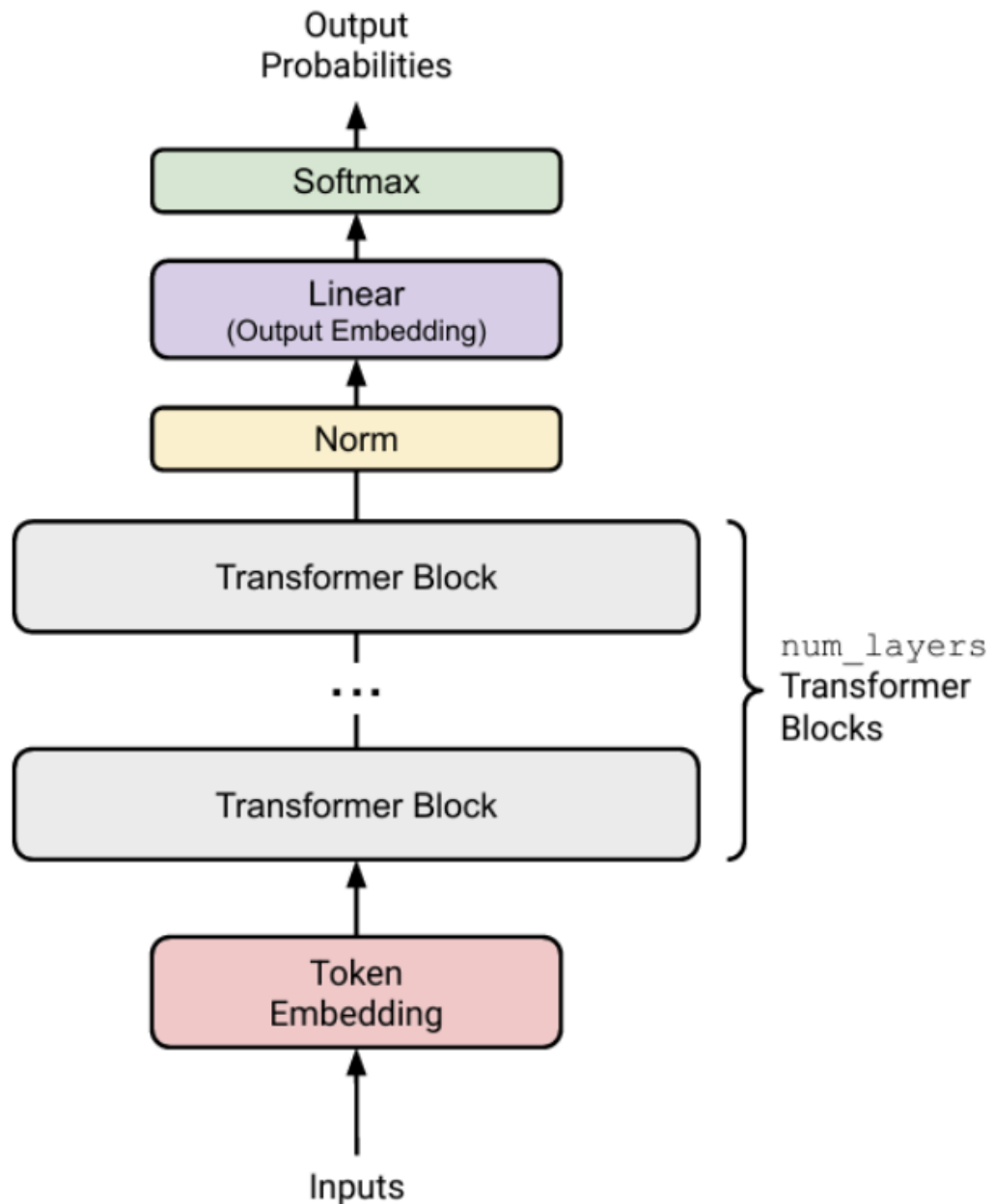
```

43 |
44 | y_norm = self.lm2(y)
45 |
46 | output = y + self.ffn(y_norm)
47 |
48 | return output

```

## Transformer 模型

- 组装模型的示意图如下
- 



- 仍然是依次组装即可
- 注意最后的结果不需要softmax, 非常奇怪
- 复习一下
  - Embedding层(batch\_size, seq\_len) -> (batch\_size, seq\_len, d\_model)



- 每个block (batch\_size, seq\_len, d\_model) -> (batch\_size, seq\_len, d\_model)
- Norm层 (batch\_size, seq\_len, d\_model) -> (batch\_size, seq\_len, d\_model)
- Linear层, 也就是head模块, 将结果映射回vocab, 实现预测 (batch\_size, seq\_len, d\_model) ->(batch\_size, seq\_len, vocab\_size)

```

1 class TransformerLanguageModel(nn.Module):
2 def __init__(self,
3 vocab_size,
4 context_length,
5 d_model,
6 num_layers,
7 num_heads: int,
8 d_ff: int,
9 rope_theta: float,
10 device: torch.device | None = None,
11 dtype: torch.dtype | None = None
12):
13 super().__init__()
14 self.vocab_size = vocab_size
15 self.context_length = context_length
16 self.num_layers = num_layers
17 self.d_model = d_model
18 self.num_heads = num_heads
19 self.d_ff = d_ff
20 self.rope_theta = rope_theta
21
22 self.embedding =
Embedding(vocab_size, d_model, device=device, dtype=dtype)
23 self.transformer_blocks = nn.ModuleList([
24 TransformerBlock(
25 d_model,
26 num_heads,
27 d_ff,
28 context_length,
29 rope_theta,
30 device=device,
31 dtype=dtype
32) for _ in range(num_layers)
33])
34 self.lm = RMSNorm(d_model, device=device, dtype=dtype)
35 self.head =
Linear(d_model, vocab_size, device=device, dtype=dtype)
36
37 def forward(self, token_ids: torch.Tensor) -> torch.Tensor:

```

```

38 x = self.embedding(token_ids) # (batch_size, seq_len,
 d_model)
39 for block in self.transformer_blocks:
40 x = block(x) # (batch_size, seq_len, d_model)
41 x = self.lm(x) # (batch_size, seq_len, d_model)
42 logits = self.head(x) # (batch_size, seq_len,
 vocab_size)
43 # 为什么这里不用softmax?????
44 return logits

```

## Train-Loop的实现

### 原理

主要是实现训练过程所需要的三大工具

- 损失函数CrossEntropy
- 优化器AdamW, 包括学习率调度和梯度截断
- Train Loop的工具, 包括load\_data, save\_model, load\_model等

根据原文档中给出的公式和伪代码实现, 注意实现时trick很多, 最好使用torch的库函数, 否则很有可能导致出现大量NaN错误

### 交叉熵公式

standard cross-entropy (negative log-likelihood) loss function:

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} | x_{1:i}). \quad (16)$$

(Note that a single forward pass in the Transformer yields  $p_{\theta}(x_{i+1} | x_{1:i})$  for all  $i = 1, \dots, m$ .)

In particular, the Transformer computes logits  $o_i \in \mathbb{R}^{\text{vocab\_size}}$  for each position  $i$ , which results in:

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab\_size}} \exp(o_i[a])}. \quad (17)$$

**Deliverable:** Write a function to compute the cross entropy loss, which takes in predicted logits ( $o_i$ ) and targets ( $x_{i+1}$ ) and computes the cross entropy  $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ . Your function should handle the following:

### 混淆度公式

**Perplexity** Cross entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length  $m$  where we suffer cross-entropy losses  $\ell_1, \dots, \ell_m$ :

$$\text{perplexity} = \exp \left( \frac{1}{m} \sum_{i=1}^m \ell_i \right). \quad (18)$$

### AdamW优化器伪代码

---

**Algorithm 1** AdamW Optimizer

---

```
init(θ) (Initialize learnable parameters)
 $m \leftarrow 0$ (Initial value of the first moment vector; same shape as θ)
 $v \leftarrow 0$ (Initial value of the second moment vector; same shape as θ)
for $t = 1, \dots, T$ do
 Sample batch of data B_t
 $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$ (Compute the gradient of the loss at the current time step)
 $m \leftarrow \beta_1 m + (1 - \beta_1)g$ (Update the first moment estimate)
 $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$ (Update the second moment estimate)
 $\alpha_t \leftarrow \alpha \frac{\sqrt{1 - (\beta_2)^t}}{1 - (\beta_1)^t}$ (Compute adjusted α for iteration t)
 $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v} + \epsilon}$ (Update the parameters)
 $\theta \leftarrow \theta - \alpha \lambda \theta$ (Apply weight decay)
end for
```

---

## 学习率调度器

A scheduler is simply a function that takes the current step  $t$  and other relevant parameters (such as the initial and final learning rates), and returns the learning rate to use for the gradient update at step  $t$ . The simplest schedule is the constant function, which will return the same learning rate given any  $t$ .

The cosine annealing learning rate schedule takes (i) the current iteration  $t$ , (ii) the maximum learning rate  $\alpha_{\max}$ , (iii) the minimum (final) learning rate  $\alpha_{\min}$ , (iv) the number of *warm-up* iterations  $T_w$ , and (v) the number of cosine annealing iterations  $T_c$ . The learning rate at iteration  $t$  is defined as:

(Warm-up) If  $t < T_w$ , then  $\alpha_t = \frac{t}{T_w} \alpha_{\max}$ .

(Cosine annealing) If  $T_w \leq t \leq T_c$ , then  $\alpha_t = \alpha_{\min} + \frac{1}{2} \left( 1 + \cos \left( \frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min})$ .

(Post-annealing) If  $t > T_c$ , then  $\alpha_t = \alpha_{\min}$ .

## 梯度截断

Given the gradient (for all parameters)  $g$ , we compute its  $\ell_2$ -norm  $\|g\|_2$ . If this norm is less than a maximum value  $M$ , then we leave  $g$  as is; otherwise, we scale  $g$  down by a factor of  $\frac{M}{\|g\|_2 + \epsilon}$  (where a small  $\epsilon$ , like  $10^{-6}$ , is added for numeric stability). Note that the resulting norm will be just under  $M$ .

注意梯度截断这里不太好理解，以防你忘记了——梯度是一个标量，是所有参数偏导数的和，这里最好结合代码看

## 其他模块

没有很重要的原理

## 思考

交叉熵和AdamW一定是重点，需要能够熟练手撕代码

其他的模块感觉并不是很重要，大概知道是怎么实现的就行

## 实现

### CrossEntropy

- 注意这个公式，需要知道发生了什么
- 首先对输出序列O的最后一维进行log-softmax运算
- 再按照标签索引进行负对数似然

○

$$\ell(\theta; D) = \frac{1}{|D|m} \sum_{x \in D} \sum_{i=1}^m -\log p_{\theta}(x_{i+1} \mid x_{1:i}).$$

- **以防你不知道这句话在说什么**——targets是标准的“下一个token”，而传入交叉熵函数的是对这个token在vocab范围内的预测，该预测在大小为 vocab\_size 的范围内给出属于每一个token的概率，概率之和为1。因此，这里通过标签索引得到了正解的概率的负对数（概率都小于1，求对数之后小于零，取负得到交叉熵），对结果求平均就是交叉熵函数

- logits的形状(batch\_size, vocab\_size)

- target的形状(batch\_size, )

- ```
1 def CrossEntropyLoss(logits: torch.Tensor, targets:
  torch.Tensor) -> torch.Tensor:
2     # 数值稳定: 先 log_softmax, 再按标签索引负对数似然
3     log_probs = logits.log_softmax(dim=-1)
4     loss = -
  log_probs[torch.arange(logits.shape[0]), targets].mean()
5     return loss
```

AdamW

- 中文描述一下AdamW优化器干了什么事

○ 循环外

- 初始化所有可学习的参数，一般是模块中的权重矩阵
- 和参数同形状的动量m
- 和参数同形状的动量n
- 读入超参数，包括学习率lr，权重衰减率weight_decay，控制动量影响的beta1和beta2，防止分母过小的eps

○ 循环内

- 先计算所有参数的梯度
- 根据beta1和梯度更新第一个动量
- 根据beta2和梯度的平方更新第二个动量
- 根据beta1, beta2, 超参数alpha以及当前训练轮数t更新当前轮数的学习率alpha_t
- 应用alpha_t, 两个动量向量m和v来更新参数（**加入动量后，参数更新就可以保持之前更新趋势，而不会卡在当前梯度较小的点了**）
- 应用超参数alpha和lambda，对参数进行权重衰减（**权重衰减可以降低模**

型复杂度)

- **for** $t = 1, \dots, T$ **do**
 - Sample batch of data B_t
 - $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$ (Compute the gradient of the loss at the current time step)
 - $m \leftarrow \beta_1 m + (1 - \beta_1)g$ (Update the first moment estimate)
 - $v \leftarrow \beta_2 v + (1 - \beta_2)g^2$ (Update the second moment estimate)
 - $\alpha_t \leftarrow \alpha \frac{\sqrt{1 - (\beta_2)^t}}{1 - (\beta_1)^t}$ (Compute adjusted α for iteration t)
 - $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v + \epsilon}}$ (Update the parameters)
 - $\theta \leftarrow \theta - \alpha \lambda \theta$ (Apply weight decay)

- 为什么能实现这些功能
 - torch中优化器的工作分为三部分：保存需要更新的参数、保存优化超参数、管理每个参数的状态
 - `torch.optim.Optimizer` 能够在过程中保存一些参数传递给自己，通过 `self.state` 来访问这些状态，一般是训练的轮数等数据
 - 除此之外，在初始化时可以通过构造一个字典的方式将参数传入对象中，通过 `self.param_groups` 访问
- 在初始化时，外部传入需要优化的参数params和超参数，在init方法中，传入params和defaults从而使得torch能够识别并将其注册为一个优化器
- 为什么要这样配置
 - torch通过这样配置来实现为不同的params捆绑不同的超参数，也就是说，每次init方法传入的params和defaults被作为param_groups的一项，在后续进行优化时也可以通过group访问，例如params就是 `group["params"]`，其他的超参数也通过类似的方法访问
- 参数的访问可以通过group来完成，训练过程中的状态则通过 `self.state` 来保存并更新
 - 同样类似一个字典，用get方法来获取参数，需要设置get默认值，防止初次访问keyError
 - 在优化时，主要是保存当前训练轮数t，两个动量向量m和v，在该轮优化结束时需要将修改过的

- ```
1 class AdamW(torch.optim.Optimizer):
2 def __init__(self, params, lr=1e-3, weight_decay=0.01, betas=
3 (0.9, 0.999), eps=1e-8):
4 if lr < 0.0:
5 raise ValueError("Invalid learning rate:
6 {}").format(lr))
7 if not 0.0 <= betas[0] < 1.0:
8 raise ValueError("Invalid beta parameter:
9 {}").format(betas[0]))
10 if not 0.0 <= betas[1] < 1.0:
```

```

8 raise ValueError("Invalid beta parameter:
{}".format(betas[1]))
9 if not 0.0 <= eps:
10 raise ValueError("Invalid epsilon value:
{}".format(eps))
11 if not 0.0 <= weight_decay:
12 raise ValueError("Invalid weight_decay value:
{}".format(weight_decay))
13 defaults = {
14 "lr": lr,
15 "betas": betas,
16 "eps": eps,
17 "weight_decay": weight_decay,
18 }
19 super().__init__(params, defaults)
20
21 def step(self, closure: Optional[Callable] = None):
22 loss = None if closure is None else closure()
23 for group in self.param_groups:
24 lr = group["lr"]
25 weight_decay = group["weight_decay"]
26 beta1, beta2 = group["betas"]
27 eps = group["eps"]
28 for p in group["params"]:
29 if p.grad is None:
30 continue
31 state = self.state[p]
32 t = state.get("t", 0)
33 t = t + 1
34
35 grad = p.grad.data
36 m = state.get("m", torch.zeros_like(grad))
37 v = state.get("v", torch.zeros_like(grad))
38 m = beta1 * m + (1-beta1) * grad
39 v = beta2 * v + (1-beta2) * (grad**2)
40 at = lr * (math.sqrt(1 - (beta2 ** t))) / (1 -
(beta1 ** t))
41 p.data -= at * m / (torch.sqrt(v) + eps)
42 p.data *= (1 - lr * weight_decay)
43 state["t"] = t
44 state["m"] = m
45 state["v"] = v
46 return loss

```

## LR Cosine Schedule

- 按照以下描述对学习率进行动态调度，可以在刚开始训练时学习率较大、快速收敛，在快结束时学习率较小，稳定找到最优值
- (Warm-up) If  $t < T_w$ , then  $\alpha_t = \frac{t}{T_w} \alpha_{\max}$ .  
(Cosine annealing) If  $T_w \leq t \leq T_c$ , then  $\alpha_t = \alpha_{\min} + \frac{1}{2} \left( 1 + \cos \left( \frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min})$ .  
(Post-annealing) If  $t > T_c$ , then  $\alpha_t = \alpha_{\min}$ .
- 其中 $T_w$ ,  $T_c$ ,  $\alpha_{\min}$ ,  $\alpha_{\max}$ 都是外部传入的参数
- 刚开始训练时，在warm-up阶段学习率逐渐到达 $\alpha_{\max}$
- 之后进入余弦退火cosine annealing阶段，学习率从 $\alpha_{\max}$ 逐渐退化到 $\alpha_{\min}$
- 在退火后post-annealing阶段，学习率固定为 $\alpha_{\min}$

```
1 def get_lr_cosine_schedule(
2 it: int,
3 max_learning_rate: float,
4 min_learning_rate: float,
5 warmup_iters: int,
6 cosine_cycle_iters: int,
7):
8 if it < warmup_iters:
9 return it/warmup_iters * max_learning_rate
10 elif warmup_iters <= it <= cosine_cycle_iters:
11 return min_learning_rate + (1 + math.cos((it -
warmup_iters)/(cosine_cycle_iters - warmup_iters) * math.pi)) *
(max_learning_rate - min_learning_rate) / 2
12 else:
13 return min_learning_rate
```

## Gradient Clipping

- 防止梯度爆炸，在RNN和LSTM中很常见
- 通过L2范数的大小来衡量，具体到计算过程中就是所有参数梯度向量的L2范数平方和的平方根，当L2范数超过某个阈值时，等比例缩小所有梯度
- 注意，如果有参数没有梯度，一定要跳过
- 通过 `p.grad.data` 可以获得一个参数的梯度的tensor，通过 `.norm(2)` 方法可以计算该张量的L2范数，最后通过 `.item()` 转换为浮点数

o

$$\text{param\_norm} = \sqrt{\sum_i (g_i)^2}$$

- 最后计算整体的范数，就是所有参数范数的平方和再开平方根

o 所以公式其实应该长这样 
$$\text{total\_norm} = \sqrt{\sum_p \|g_p\|_2^2}$$

- 最后根据范数是否超出阈值，对所有梯度进行对应的截断

```
1 def gradient_clipping(parameters:
 Iterable[torch.nn.Parameter], max_l2_norm: float):
2 total_norm = 0.0
3 for p in parameters:
4 if p.grad is None:
5 continue
6 param_norm = p.grad.data.norm(2).item()
7 total_norm += math.pow(param_norm, 2)
8 total_norm = math.sqrt(total_norm)
9 coef = max_l2_norm / (total_norm + 1e-6)
10 if coef < 1:
11 for p in parameters:
12 if p.grad is None:
13 continue
14 p.grad.data.mul_(coef)
15 return parameters
```

## Get Batch

- 首先确定抽样的范围，给出的dataset是一段一维的id序列，batch\_size是需要生成的样本的数量，context\_length是生成的一条样本的长度
- 由于需要保证所有样本等长，所以能作为样本首个id的下标范围在[0:len(dataset) - context\_length) 中，**注意不包含右边界**，提取出这些样本进行随机打乱，抽出其中前batch\_size个样本

```
1 def get_batch(
2 dataset: npt.NDArray, batch_size: int, context_length:
 int, device: str
3) -> tuple[torch.Tensor, torch.Tensor]:
4 full_lenth = len(dataset)
5 indexs = [i for i in range(full_lenth - context_length)]
6 random.shuffle(indexs)
```



```

7 pre_starts = indexes[:batch_size]
8 pre_tokens = []
9 next_tokens = []
10 for i in range(batch_size):
11
12 pre_tokens.append(dataset[pre_starts[i]:pre_starts[i]+context
13 _length])
14 next_tokens.append((dataset[pre_starts[i] +
15 1:pre_starts[i]+context_length +1]))
16 pre_tokens_np = np.array(pre_tokens)
17 next_tokens_np = np.array(next_tokens)
18 return torch.from_numpy(pre_tokens_np).to(device),
19 torch.from_numpy(next_tokens_np).to(device)

```

## Load/Save Checkpoint

- 其实就是保存/读取模型的参数，`state_dict()` 方法保存了当前对象中所有可学习参数和缓冲区的状态，收集模型、优化器和当前训练轮数的数据，构造一个字典，并将其通过 `torch.save()` 方法保存在指定路径中
- 读取时，应用 `torch.load()` 方法读入一个字典，将对应的项按名索引，并使用 `load_state_dict()` 方法加载到参数中

- ```

1  def save_checkpoint(
2      model:torch.nn.Module,
3      optimizer:torch.optim.Optimizer,
4      iteration:int,
5      out:str|os.PathLike |typing.BinaryIO
6      |typing.IO[bytes]):
7      model_dict = model.state_dict()
8      optim_dict = optimizer.state_dict()
9      d = {
10          "model_dict":model_dict,
11          "optim_dict":optim_dict,
12          "iteration":iteration,
13      }
14      torch.save(d,out)
15
16  def load_checkpoint(
17      src:str|os.PathLike |typing.BinaryIO
18      |typing.IO[bytes],
19      model:torch.nn.Module,
20      optimizer:torch.optim.Optimizer
21  ):
22      d = torch.load(src)
23      model_dict = d["model_dict"]

```

```
22     optim_dict = d["optim_dict"]
23     model.load_state_dict(model_dict)
24     optimizer.load_state_dict(optim_dict)
25     return d["iteration"]
```